

P1 - Reliable Batch Pipeline

ORDERS + PAGINATED API + POSTGRESQL | PROJECT BOOK

Business story in beginner language

A small marketplace receives orders from CSV, customers from a paginated API and another operational source. The danger is not "can Python read a file?" The danger is duplicate deliveries, bad records, partial API success and reruns that silently duplicate trusted data.

What you are building

One repeatable batch pipeline with raw evidence, validation/quarantine, reconciliation, safe PostgreSQL trusted load, tests/logging, rerun/backfill and one repaired incident.

What this project is really testing

- Multi-source ingestion with clear contracts.
- Raw vs staging vs trusted boundaries.
- Row/business-key reasoning and reconciliation.
- Idempotent reruns and partial-failure recovery.
- Configuration/secrets hygiene, logging and automated tests.
- Architecture/runbook/technical defense.

Important: teaching demo vs your protected project

The mini example below teaches the pattern using different toy data. It is NOT the final solution for your supplied project. After you understand the pattern, close the example and apply the reasoning to the project assets yourself.

Tiny teacher demonstration

Toy case: one CSV plus two API pages

Imagine orders.csv contains O1/O2/O2 and API customer pages contain C1/C2/C3. First preserve inputs, then deduplicate/validate according to business keys. Do not load directly into trusted tables before you can explain where rejected/duplicate rows went.

```
raw/batch_20260820/  
orders.csv  
api_page_1.json  
api_page_2.json  
metadata.json  
metadata example:  
{  
  "batch_id": "20260820T0600",  
  "source_file": "orders.csv",  
  "received_at": "...",  
  "sha256": "..."  
}
```

Raw preservation means you can later reproduce what the source actually sent. Trusted tables are built from parsed/validated records, not by modifying raw files.

Source outcome	Example count
source CSV rows	5
exact duplicate removed	1

Source outcome	Example count
invalid required-key/date/amount	1
trusted orders	3

The reconciliation is $5 = 1 \text{ deduplicated} + 1 \text{ quarantined} + 3 \text{ trusted}$. Your real P1 has its own expected control totals; the pattern is what matters.

Paginated API in plain English

One HTTP/API response may not contain all customers. Page 1 says where page 2 is. Your ingestion loop continues until there is no next page. Save each raw page before transforming it.

```

page = 1
while page exists:
    payload = fetch(page)
    save_raw(payload, page)
    validate/collect records
    page = payload.next_page

```

STOP - explain before continuing

If page 2 fails after page 1 was saved, why should the pipeline NOT publish a fake "complete" customer snapshot?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Your project build order

Phase 1 - Read and profile the supplied sources

First understand

You need to know what each file/page/table represents before writing ingestion. P1 mixes CSV, paginated API pages and a database seed deliberately.

Then do it in your project

1. List every source file/table/API page.
2. Write each source grain and business key.
3. Profile missing IDs, duplicate IDs, bad dates and negative amounts.
4. Write your first reconciliation expectations.

Evidence gate - prove before move-on

- source profile saved
- grain/key written for every source
- known bad-data examples identified

STOP - explain before continuing

Before Phase 2: can you explain what changed in the system and what evidence proves it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Phase 2 - Create raw landing + batch metadata

First understand

Raw is evidence. Preserve source payloads/files before cleanup so reruns/debugging can reproduce the original batch.

Then do it in your project

1. Create raw folder naming/batch metadata strategy.
2. Copy/save source input without modifying it.
3. Record batch/source metadata such as batch_id, filename/page, received time and optional checksum.

Evidence gate - prove before move-on

- raw evidence exists
- metadata ties records/files to batch
- raw not overwritten during transformation

STOP - explain before continuing

Before Phase 3: can you explain what changed in the system and what evidence proves it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Phase 3 - Build CSV validation + quarantine

First understand

Required key/date/amount rules should produce explicit accepted vs rejected outcomes.

Then do it in your project

1. Parse orders.csv.
2. Reject missing order_id, invalid date and negative amount with reason.
3. Decide duplicate-row behavior using business key/record semantics.
4. Write reconciliation summary.

Evidence gate - prove before move-on

- source = accepted + quarantined + deduplicated as designed
- quarantine has reason per rejected row
- trusted grain remains one row per accepted business order

STOP - explain before continuing

Before Phase 4: can you explain what changed in the system and what evidence proves it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Phase 4 - Build paginated API ingestion

First understand

Pagination is a control-flow problem: fetch/save/process pages until completion; partial failure must be visible.

Then do it in your project

1. Read page1/page2 JSON contract.
2. Implement page loop/next-page rule.
3. Preserve each page raw.
4. Handle malformed record separately from transport/page failure.

Evidence gate - prove before move-on

- all expected pages processed in healthy run
- page-level raw evidence saved
- partial-page failure prevents misleading complete publish

STOP - explain before continuing

Before Phase 5: can you explain what changed in the system and what evidence proves it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Phase 5 - Add PostgreSQL staging/trusted load

First understand

Staging is receiving/validation boundary; trusted load must use the project business key and rerun rule.

Then do it in your project

1. Create/describe staging and trusted schema.
2. Load accepted data to staging.
3. Implement safe insert/upsert/merge logic.
4. Separate configuration from credentials.

Evidence gate - prove before move-on

- trusted row grain documented
- same-key rerun behavior proven
- no plaintext secret in code

STOP - explain before continuing

Before Phase 6: can you explain what changed in the system and what evidence proves it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Phase 6 - Add logging + tests

First understand

Logs tell you which batch/source/phase failed. Tests protect duplicate, missing ID, bad date, negative amount and rerun rules.

Then do it in your project

1. Add structured/sensible logging.
2. Implement tests named for failure behavior.
3. Run tests before and after failure repair.

Evidence gate - prove before move-on

- test output saved
- logs identify batch/phase without secrets
- critical rules have automated checks

STOP - explain before continuing

Before Phase 7: can you explain what changed in the system and what evidence proves it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Phase 7 - Prove idempotency + backfill

First understand

Rerun is a behavior you must execute, not a README adjective.

Then do it in your project

1. Run same healthy batch once and record trusted count/amount.
2. Run same logical batch again.
3. Compare controls.
4. Write bounded backfill/recovery procedure.

Evidence gate - prove before move-on

- trusted logical count does not double
- trusted amount does not double
- runbook names exact rerun/backfill steps

STOP - explain before continuing

Before Phase 8: can you explain what changed in the system and what evidence proves it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Phase 8 - Trigger failure pack and repair

First understand

A portfolio project becomes stronger when you prove partial API/malformed/redelivery recovery.

Then do it in your project

1. Choose supplied failure scenario.
2. Save first failing evidence.
3. Name root cause.
4. Apply smallest repair.
5. Rerun failed path + full controls.

Evidence gate - prove before move-on

- before/after evidence saved
- trusted publish remains safe
- repair still safe on another rerun

STOP - explain before continuing

Before Phase 9: can you explain what changed in the system and what evidence proves it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Phase 9 - Finish documentation + defense

First understand

Only document what you actually ran/proved.

Then do it in your project

1. Complete README, architecture.md and runbook.md.
2. Add known limitations.
3. Run project checker.
4. Practice defense prompts without reading a script.

Evidence gate - prove before move-on

- checker/tests saved
- README run steps match reality
- architecture shows raw/staging/trusted/failure path

STOP - explain before continuing

Before Phase complete: can you explain what changed in the system and what evidence proves it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Expected control evidence

Control	Expected from supplied starter/checker
source_rows	8
dedup_rows	7
valid_orders	4
quarantined_orders	3
valid_amount	5800.0
customers_loaded	7
payments_loaded	4

Deliberate failures you must practise

Repeated input / duplicate redelivery

The same business content can arrive twice.

Save first failing evidence before repair.

- prove trusted logical order count/totals do not double
- explain raw retention vs trusted dedup

Malformed API record

One record/page contains invalid structure.

Save first failing evidence before repair.

- save raw malformed evidence
- quarantine/fail according to contract
- do not silently drop required fields

Partial API failure

A later page fails after earlier pages succeeded.

Save first failing evidence before repair.

- save partial failure log
- prevent false complete publish
- show safe retry/resume policy

Technical defense

What is the grain of your trusted orders output?

Your answer: _____

Evidence you would point to: _____

What exact key/rule makes reruns safe?

Your answer: _____

Evidence you would point to: _____

Where do rejected rows go and how do source/trusted counts reconcile?

Your answer: _____

Evidence you would point to: _____

If API page 2 fails after page 1, what persists and what is blocked?

Your answer: _____

Evidence you would point to: _____

What would you change at 10x data volume?

Your answer: _____

Evidence you would point to: _____

Which local simplification is least production-ready?

Your answer: _____

Evidence you would point to: _____

Honest limitations

A local portfolio project does not prove high availability, production IAM, distributed scale or managed database operations. State that clearly; then explain what would change if those requirements existed.

Evidence gate - prove before move-on

- README explains what is local/simulated.
- No claim of production scale without evidence.
- You can say what you would change at higher volume/availability/security requirements.